

Технички извештај — MQTT наспрам Kafka: IoT микросервиси засновани на догађајима

Интернет ствари и сервиса — Пројекат 2 (2026)

Аутори: Андрија Грбушић, Александар Митић

Овај документ представља званичну српску (ћириличну) верзију извештаја из `docs/report.md`, потпуно попуњену стварним резултатима мерења из емпиријских сценарија А–Д, за оба брокера и све тестиране конфигурације поузданости (QoS и `acks`). Последње поглавље доноси ригорозну проверу усклађености са Спецификацијом пројекта, са критичким освртом на методолошке изборе.

1. Преглед система и архитектуре

Пројектовани систем имплементира асинхрону, догађајима вођену (event-driven) микросервисну архитектуру за прихват, складиштење и стрим аналитику IoT телеметријских података у реалном времену. Комплетан систем је контејнеризован помоћу Docker Compose-а и омогућава потпуну независност сервисних компоненти од конкретног message broker-а захваљујући променљивој окружења `BROKER_MODE=mqtt|kafka`. У складу са захтевима спецификације, систем је развијен коришћењем две софтверске технологије (Node.js и .NET / ASP.NET Core).

Архитектура се састоји од три кључна микросервиса чије су улоге детаљно описане у наставку:

Сервис	Технологија	Улога и имплементациона спецификација
ingestion	Node.js	Симулира конзистентан паралелни рад N IoT уређаја и емитује читавања сензора квалитета ваздуха (користећи Air Quality dataset из Пројекта 1) на активни брокер. Параметри поузданости (QoS за MQTT, односно <code>acks</code> за Кафку) су динамички подесиви.
storage	.NET (ASP.NET Core)	Претплаћен на активни ток података, преузима поруке и групно их уписује (batching) у PostgreSQL базу на сваких 500 порука (<code>STORAGE_BATCH_SIZE=500</code>) како би се спречило да дисковни I/O подсистем постане уско грло.
analytics	Node.js	Врши Stream Processing над истим током података. Имплементира Tumbling Window од 10 секунди. Рачуна просек температуре и генерише [ALERT] у логу ако просек прозора пређе дефинисани праг (<code>ALERT_THRESHOLD=50°C</code>). Такође мери енд-то-енд кашњење сваке поруке.

Спецификација Message Broker конфигурација

- Mosquitto (MQTT брокер):** Подржава нивое квалитета услуге QoS 0, QoS 1 и QoS 2. За сервис `analytics` конфигурирана је трајна (persistent) сесија постављањем параметра `clean: false` како би се пратио

опоравак порука. Датотека `mosquitto.conf` експлицитно укључује персистенцију на диску (`persistence true`), као и лимите `max_queued_messages 10000` и `max_inflight_messages 100`.

- **Apache Kafka (KRaft режим):** Архитектура ради без Zookeeper-a, чиме се значајно штеде меморијски ресурси. Топик `iot.readings` је креиран са 3 партиције (`KAFKA_NUM_PARTITIONS=3`). Конфигурисане су две независне consumer групе које паралелно читају податке: `project-two-storage` и `analytics-group`. Омогућено је тестирање за вредности `acks=0`, `acks=1` и `acks=all`.

2. Експериментални IoT сценарији

Како би се извршила детаљна компаративна евалуација, систем је изложен кроз четири ригорозна емпиријска сценарија, дефинисана аутоматизованим benchmark скриптама. Оптерећење се покреће и контролише путем HTTP API-ја на `ingestion` сервису (путе `/simulate/start` и `/simulate/stop`).

Сценарио	Опис и циљ мерења	Путања бенчмарк скрипте	Примарне метрике
Scenario A	Масовни унос: 100 / 1000 / 10000 симулираних уређаја @ 1 msg/s, у трајању од 30s по нивоу.	<code>benchmarks/{broker}/scenario_a.sh</code>	Throughput (msg/s), проценат изгубљених порука у бази података.
Scenario B	Прекид конекције на edge-у: Емитовање мрежног дисконектовања из Docker мреже у трајању од 30s.	<code>benchmarks/{broker}/scenario_b.sh</code>	Анализа трајних претплата, померање offset-a и брзина опоравка.
Scenario C	Налет оптерећења (Burst): Скок са конзистентних ~50 msg/s (15s) на ~5000 msg/s (10s), па опоравак (30s).	<code>benchmarks/{broker}/scenario_c.sh</code>	Формирање backlog-a, backpressure понашање, време стабилизације.
Scenario D	Алармирање у реалном времену: Форсирање вредности температура изнад 50°C.	<code>benchmarks/{broker}/scenario_d.sh</code>	Енд-то-енд латенција аларма (генерисање → испис у лог).

3. Упоредне табеле перформанси

3.1 Сценарио А — Пропусност и губитак порука

За MQTT архитектуру, мерење је извршено искључиво на нивоу PostgreSQL базе, с обзиром на то да Mosquitto не поседује изворни концепт commit лога или offset-a из којег би се прочитало стање на самом брокеру.

Табела 1: Перформансе MQTT (Mosquitto) у Сценарију А

QoS ниво	Број уређаја	Очекивано порука	Примљено у бази (DB)	Губитак (%)	Пропусност базе (msg/s)
0	100	3 000	2 900	3.33%	96.67
1	100	3 000	2 900	3.33%	96.67
2	100	3 000	2 900	3.33%	96.67
0	1 000	30 000	29 000	3.33%	966.67
1	1 000	30 000	29 000	3.33%	966.67
2	1 000	30 000	27 691	7.70%	923.03
0	10 000	300 000	92 503	69.17%	3 083.43
1	10 000	300 000	83 701	72.10%	2 790.03
2	10 000	300 000	72 094	75.97%	2 403.13

За разлику од MQTT-а, Apache Kafka омогућава праћење на два независна нивоа: на нивоу самог брокера (метрика `delta log-end-offset`-а која показује шта је трајно персистовано на брокеру) и на нивоу `storage` сервиса у бази података.

Табела 2: Перформансе Apache Kafka (KRaft) у Сценарију А

acks	Уређаји	Очекивано	База (DB)	Throughput DB	Губитак DB	Заостатак (Lag)	Примљено брокер	Губитак брокер
0	100	3 000	2 900	96.67 msg/s	3.33%	0	≈ 2 900	≈ 3.3%
1	100	3 000	2 900	96.67 msg/s	3.33%	0	≈ 2 900	≈ 3.3%
all	100	3 000	2 901	96.70 msg/s	3.30%	0	≈ 2 901	≈ 3.3%
0	1 000	30 000	29 000	966.67 msg/s	3.33%	0	≈ 2 9000	≈ 3.3%
1	1 000	30 000	29 000	966.67 msg/s	3.33%	0	≈ 2 9000	≈ 3.3%
all	1 000	30 000	29 000	966.67 msg/s	3.33%	0	≈ 2 9000	≈ 3.3%
0	10 000	300 000	86 501	2883.37 msg/s	71.17%	196 227	285 228	4.92%
1	10 000	300 000	82 501	2750.03 msg/s	72.50%	200 431	285 432	4.86%
all	10 000	300 000	82 501	2750.03 msg/s	72.50%	200 014	284 515	5.16%

Критичка анализа резултата: Примећује се конзистентан "губитак" од ~3.33% на нижим нивоима (100 и 1000 уређаја) код оба брокера. Овај феномен одговара кашњењу од тачно 1 секунде у прозору од 30 секунди. Реч је о артефакту тестне скрипте услед асинхроног покретања HTTP руте `/simulate/start` у односу на читавање бројача у бази, а не о стварном губитку пакета на мрежи. Најрелевантнији подаци се уочавају на 10.000 уређаја, где код MQTT-а губитак расте прогресивно са повећањем QoS нивоа (од 69.17% на QoS0 до 75.97% на QoS2) због тешког 4-смерног handshake-а. Код Кафке, губитак у бази износи фиксних 71-73% без обзира на вредност `acks`, али је губитак на самом брокеру заправо занемарљив (~5%). Ово јасно доказује да је Кафка примила скоро све поруке (~9500 msg/s), али је упис у базу кроз `storage` сервис постао уско грло подсистемског I/O капацитета, што је резултовало високим Consumer Lag-ом (~200.000 порука).

3.2 Сценарио В — Опоравак након 30s мрежног прекида

Овај сценарио симулира рад 20 уређаја који емитују по 1 msg/s током укупно 90 секунди, при чему се мрежни интерфејс `ingestion` сервиса искључује између $t=15s$ и $t=45s$.

Табела 3: Динамика опоравка у Сценарију В

Брокер	Конфигурација	Вредност у $t=45s$	Вредност у $t=90s$	Опорављено порука	Први скок раста након реконекције
MQTT	QoS 0	300	1 780	1 480	+25s (y $t=70s$)
MQTT	QoS 1	300	1 780	1 480	+25s (y $t=70s$)
MQTT	QoS 2	300	820	520	+25s (y $t=70s$)
Kafka	acks = 1	300	2 240	1 940	+25s (y $t=70s$)
Kafka	acks = all	300	2 240	1 940	+25s (y $t=70s$)
Kafka	acks = 0	84 501	95 717	11 216	Одмах (расте континуирано у прекиду)

Анализа динамике опоравка: Код MQTT-а (сви QoS) и Кафке (сејф модови 1/all), графикон броја порука је потпуно раван за време трајања прекида, што показује физичку немогућност слања. Након поновног повезивања у $t=45s$, долази до латенције од ~25 секунди пре него што софтверски клијент испоручи локално баферисане поруке у једном великом налету (burst) око $t=70s$. Кафка са `acks=0` је потпуно изолована: база података наставља континуирано да расте и током самог трајања мрежног прекида. Ово је врхунски доказ потпуне сепарације слојева (producer/consumer decoupling) код Кафке — `storage` сервис неометано чита и празни наслеђени заостатак (lag) из Сценарија А директно са брокера, потпуно агностичан у односу на мрежни статус симулатора уређаја.

3.3 Сценарио С — Налет оптерећења (Burst од 50 → 5000 msg/s)

У овом тесту прати се понашање током наглог десетоструког скока оптерећења у трајању од 10 секунди (теоријски очекивани укупан обим саобраћаја износи ~50.750 порука).

Табела 4: Понашање система под наглим налетом оптерећења (Burst)

Брокер	QoS/acks	Врх backlog-a	Персистовано на крају	Трајни губитак	Време стабилизације	CPU макс (%) (ing/st/an/br)	RAM макс (MiB) (ing/st/an/br)
MQTT	0	24 549 (t=25s)	50 200	550 (1.08%)	10s	35.3 / 52.9 / 29.7 / 13.9	48.0 / 95.8 / 61.9 / 5.7
MQTT	1	27 547 (t=25s)	34 758	15 992 (31.51%)	6s (плато)*	61.8 / 59.1 / 49.1 / 20.3	76.1 / 81.1 / 62.4 / 13.5
MQTT	2	22 049 (t=25s)	40 466	10 284 (20.26%)	4s (плато)*	40.0 / 60.1 / 43.7 / 25.8	77.9 / 81.3 / 80.6 / 13.7
Kafka	0	22 049 (t=24s)	49 915	835 (1.65%)	6s	84.3 / 27.7 / 65.1 / 271.6	47.4 / 110.8 / 70.6 / 1166.3
Kafka	1	21 049 (t=24s)	49 933	817 (1.61%)	6s	106.3 / 33.5 / 111.5 / 249.7	73.4 / 110.2 / 72.2 / 1145.9
Kafka	all	19 549 (t=24s)	49 895	855 (1.68%)	6s	45.0 / 31.9 / 111.7 / 238.0	73.6 / 108.8 / 71.4 / 1161.2

* Напомена: Код MQTT QoS 1 и QoS 2 постизање платоа не означава пуни опоравак већ стабилизацију на нивоу трајног губитка порука које су одбачене унутар зачепљеног in-flight реда на клијентској страни.

3.4 Сценарио D — Енд-то-енд кашњење алармирања

У овом сценарију систем анализира брзину детекције критичних вредности ($> 50^{\circ}\text{C}$) и процесирање унутар аналитичког сервиса кроз три консекутивна временска прозора.

Табела 5: Енд-то-енд кашњење стрим аналитике (у милисекундама)

Брокер	Конфигурација	Латенција Прозор #1	Латенција Прозор #2	Латенција Прозор #3	Просечна латенција (ms)
MQTT	QoS 0	5 ms	1 ms	1 ms	2.33 ms
MQTT	QoS 1	1 ms	1 ms	1 ms	1.00 ms
MQTT	QoS 2	9 ms	2 ms	2 ms	4.33 ms
Kafka	acks = 0	4 ms	1 ms	1 ms	2.00 ms
Kafka	acks = 1	3 ms	1 ms	1 ms	1.67 ms
Kafka	acks = all	3 ms	1 ms	1 ms	1.67 ms

Образложење у вези са p95 латенцијом: Аналитички сервис имплементиран у датотеци `services/analytics/src/tumbling-window.js` унутар функције `_flush()` бележи искључиво математички просек кашњења по прозору (`avgLatency`). С обзиром на то да појединачне латенције сваке поруке нису трајно сачуване у лог фајловима унутар `docs/results/scenario_d/*.log`, **израчунавање перцентилне p95 латенције није математички могуће** на основу доступних експерименталних података. Због тога је у Табели 5 приказан комплетан ток просечних вредности кроз прозоре. Све латенције у стању мировања су изразито ниске ($< 10\text{ms}$).

3.5 Отисак ресурса унутар Docker окружења (Resource Footprint)

Поређење заузећа хардверских ресурса извршено је између стања релативног мировања и максималног стрес-теста со саобраћајем од 10.000 симулираних уређаја у Сценарију А.

Табела 6: Компаративни преглед искоришћења CPU и RAM меморије

Брокер режим	Име контејнера	CPU % (Мировање) Просек / Макс	CPU % (10.000 уређаја) Просек / Макс	RAM (Мировање) Просек / Макс (MiB)	RAM (10.000 уређаја) Просек / Макс (MiB)
MQTT	mosquitto	0.02% / 0.04%	15.10% / 29.72%	5.28 / 6.77 MiB	10.85 / 13.95 MiB
MQTT	ingestion	0.83% / 1.51%	28.55% / 40.77%	41.95 / 47.82 MiB	123.48 / 135.20 MiB
MQTT	storage	2.20% / 10.72%	23.98% / 53.60%	26.73 / 27.70 MiB	78.07 / 81.24 MiB
MQTT	analytics	0.00% / 0.00%	19.91% / 34.28%	39.77 / 47.36 MiB	82.91 / 93.05 MiB
Kafka	kafka	52.71% / 207.27%	177.21% / 438.95%	520.53 / 579.60 MiB	953.83 / 1028.10 MiB
Kafka	ingestion	5.99% / 11.02%	99.64% / 154.62%	47.96 / 52.20 MiB	347.55 / 411.90 MiB
Kafka	storage	1.92% / 4.64%	8.40% / 38.77%	48.91 / 56.45 MiB	182.53 / 214.70 MiB
Kafka	analytics	4.06% / 6.70%	55.22% / 73.49%	36.02 / 37.36 MiB	77.06 / 85.04 MiB

4. Одговори на кључна инжењерска питања

4.1 Зашто је MQTT идеалан за сам руб мреже (edge уређаје), а неадекватан за историјску аналитику великих података?

MQTT (Message Queuing Telemetry Transport) протокол је изворно дизајниран за екстремно лагану архитектуру са минималним заглављем поруке (свега 2 бајта у фиксном делу). Измерени емпиријски отисак Mosquitto брокера у стању мировања показује невероватно ниске вредности од **< 7 MiB RAM-а и мање од 0.05% заузећа процесора**. Овакав отисак омогућава покретање брокера директно на микроконтролерима или хардверски ограниченим gateway уређајима на самом рубу мреже (edge), где су пропусни опсег, меморија и енергетски ресурси критично дефицитарни. Корисник преко различитих QoS нивоа може бирати оптималан баланс кашњења и поузданости за тренутни пренос података.

Међутим, MQTT се заснива на концепту релеја ("fire-and-forget" транзит) — чим се порука испоручи тренутно активним претплатницима, она се неповратно брише са брокера. Он нема уграђен изворни концепт трајног дисковног лога нити offset-а који омогућава поновно пуштање историјских података ("data replay"). Иако `mosquitto.conf` поседује параметар `persistence true`, то служи искључиво за чување бафера унутар сесија током краткотрајних падова конекције претплатника (као у Сценарију В), а не као база података. За потребе великих података (Big Data аналитика, тренирање ML модела, историјски backfill) неопходно је имати

дистрибуирани систем за складиштење порука, те се у овом типу архитектуре MQTT мора везивати за додатне базе, што додатно повећава сложеност и уводи нова уска грла.

4.2 Зашто Kafka доминира у клауд системима високог интензитета, колика је цена њене скалабилности и да ли је реално покретати је на edge хардверу?

Apache Kafka је изграђена на дистрибуираном, репликованом, append-only commit логи високих перформанси. Поруке се трајно записују на диск и чувају унутар дефинисаног временског рока (у нашем тесту `KAFKA_LOG_RETENTION_HOURS=2`), потпуно независно од потрошача. То омогућава да десетине потпуно различитих софтверских подсистема (на пример, наш `storage` за базу и `analytics` за стриминг) читају исти извор података потпуно независним темпом користећи сопствене поинтере (offset-e). Скалабилност се постиже хоризонталним партиционисањем топика — у нашим систему, топик са 3 партиције равномерно расподељује оптерећење на потрошачке нити унутар consumer група.

Ипак, цена ове моћне архитектуре у погледу системских ресурса је енормна. Као што је приказано у Табели 6, чак и у оптимизованом KRaft режиму без Zookeeper-a, један једини Kafka контејнер већ при лаганом раду заузима преко 500 MiB RAM меморије и конзистентно троши преко 50% једног CPU језгра, што представља 70 пута већи отисак у односу на Mosquitto. Под тешким стресом са 10.000 уређаја, Кафка троши застрашујућих 1028 MiB RAM-а и чак 438.95% процесорског времена (користећи више од 4 језгра домаћина). Из тог разлога, покретање продукционе Кафке на хардверски ограниченом edge хардверу (попут Raspberry Pi класе или уређаја са мање од 1GB RAM-а) је потпуно нереално и неисплативо. Архитектонски sweet-spot Кафке је искључиво централни клауд ниво, где служи као агрегациони слој у који се сливају подаци прикупљени са ивица мреже путем лакших протокола као што је MQTT.

5. Анализа поузданости (QoS / acks)

- **MQTT QoS 0 (At most once):** Емитовање порука без икакве потврде од стране брокера. Испољава најниже укупно заузеће процесора. Под масовним наглим burst оптерећењем у Сценарију Ц, ова конфигурација је остварила најбољи укупни резултат у MQTT групи са свега 1.08% губитка, јер Node.js клијент није губио време на handshake већ је директно избацивао телеметрију на мрежу.
- **MQTT QoS 1 (At least once):** Гарантује испоруку преко PUBACK повратне поруке, али уводи ризик од дуплирања. Доживео је потпуни колапс под наглим burst-ом у Сценарију Ц са **најгорим резултатом у читавом тесту од 31.51% изгубљених порука**. Узрок је препуњавање интерног клијентског бафера услед чекања потврда преко лимитираног броја паралелних веза.
- **MQTT QoS 2 (Exactly once):** Испуњава највиши ниво сигурности кроз компликовани 4-смерни handshake (PUBLISH/PUBREC/PUBREL/PUBCOMP). Очекивано, има највећу цену латенције у Сценарију D (4.33ms у просеку). У Сценарију А са 10.000 уређаја показао је највећи пад перформанси (пропусност базе пада на 2403.13 msg/s) због огромног софтверског overhead-а на самом Mosquitto брокеру.
- **Kafka acks = 0:** Произвођач шаље поруку и одмах наставља рад, не чекајући никакву потврду од Кафке. У Сценарију А приказао је највећи измјерени throughput ка бази података (2883.37 msg/s), али и највећи стварни губитак података на самом брокеру (4.92%).
- **Kafka acks = 1:** Клијент чека потврду само од вође партиције (Leader реплика). Овај мод нуди изврстан и избалансиран однос латенције, брзине и сигурности, бележећи најнижи лог губитак на брокеру (4.86%).
- **Kafka acks = all:** Захтева синхрону потврду уписа од свих тренутно активних реплика (In-Sync Replicas). У овом конкретном пројекту коришћен је локални појединачни брокер (single-broker са replication factor = 1), те

је режим `all` функционално био идентичан моду `acks=1`. Разлика у губитку од свега 0.30% представља уобичајени статистички шум при мерењу.

6. Закључак и инжењерске препоруке

- Ниско и средње оптерећење (100–1000 уређаја):** На овом нивоу саобраћаја, разлика између два брокера је потпуно незаметна на спољашњем слоју. Коначна архитектонска одлука треба да буде донета искључиво на основу захтева за оперативном једноставношћу и расположивих хардверских ресурса.
- Екстремни прихват података (10.000 уређаја):** Емпиријски је доказано да масивни пад перформанси у бази података (губитак између 69% и 76%) није изазван немогућношћу рада брокера. Кафка је успешно прихватила чак 95% саобраћаја. **Суштинско уско грло целог система јесте пропусна моћ паралелног масовног уписа у PostgreSQL базу података** кроз .NET сервис, упркос имплементираној batching стратегији од 500 порука. Препоручује се прелазак на специјализоване Time-Series базе података попут TimescaleDB или InfluxDB за овакве IoT системе.
- Управљање наглим скоковима (Burst):** За телеметријске системе који шаљу периодичне податке отпорне на ситне губитке (нпр. сензори квалитета ваздуха), **MQTT QoS 0 се показао као драстично супериорније решење под burst оптерећењем у односу на QoS 1 и 2**, јер у потпуности спречава појаву клијентских блокада на изворишту. Кафка показује максималну стабилност (~1.6% губитка) у свим модовима због изворног групног баферисања у меморији клијента.

7. Провера усклађености са спецификацијом пројекта

Извршена је мапирана верификација усклађености изграђеног софтверског система са захтевима из Спецификације пројекта.

Табела 7: Матрица усклађености изграђеног решења са захтевима пројекта

Ставка	Експлицитни захтев из спецификације	Статус	Доказ имплементације и коментар
Захтев 1	Исти IoT dataset/model из Пројекта 1. Docker Compose контејнеризација. Најмање две софтверске технологије.	✓ Усклађено	Коришћен Air Quality (UCI) скуп података унутар директоријума <code>db/dataset/</code> . Имплементиран <code>docker-compose.yml</code> . Компоненте развијене у Node.js и .NET технологијама.
Захтев 2	Развој 3 микросервиса (Ingestion, Storage, Analytics). Имплементација batching уписа од 500 порука. Креирање 10s Tumbling Window агрегације са алармним прагом изнад 50°C.	✓ Усклађено	Поставка са три одвојена контејнера. Конфигурисана опција <code>STORAGE_BATCH_SIZE=500</code> (класа <code>StorageOptions.cs</code>). Скрипта <code>tumbling-window.js</code> израчунава просек на 10s и исписује ознаку <code>[ALERT]</code> .
Захтев 3а	MQTT евалуација кроз промене нивоа квалитета услуге (QoS 0, QoS 1, QoS 2) и анализа ефеката на латенцију и губитак.	✓ Усклађено	Комплетан сет мерења спроведен у Сценаријима А и Ц за губитак, као и у Сценарију D за анализу кашњења порука.
Захтев 3б	Кафка евалуација у KRaft режиму. Промена <code>acks</code> параметара (0, 1, all). Детаљна анализа Consumer Lag-а и предности партиционисања.	✓ Усклађено	Параметри <code>broker, controller</code> постављени унутар <code>kafka.env</code> . Креиране 3 партиције. Ефекти заостатка (Lag) детаљно су квантификовани и образложени у секцији 3.1 извештаја.
Захтев 4	Успешно извршавање сва 4 дефинисана експериментална сценарија (A, B, C, D).	✓ Усклађено	Све бенчмарк скрипте су успешно покренуте за све конфигурације брокера. Лог фајлови са резултатима налазе се у <code>docs/results/</code> .
Захтев 5а	Обавезно бележење потрошње системских ресурса (CPU, RAM, мрежа) путем алата <code>docker stats</code> .	✓ Усклађено	Метрике су екстраховане и персистоване у форми <code>*_dockerstats.csv</code> извештаја унутар фолдера са резултатима.
Захтев 5ц/д	Обавезно коришћење наменских алата за генерисање оптерећења: <code>emqtt-bench</code> или <code>k6</code> (за MQTT), односно нативне скрипте или <code>k6 xk6-kafka</code> (за Кафку).	⚠ Одступање	Уочено одступање од слова спецификације: Уместо екстерних алата (<code>k6/emqtt-bench</code>), за генерисање и слање саобраћаја искоришћен је сопствени, прилагођени <code>ingestion</code> микросервис којим се управља

Ставка	Експлицитни захтев из спецификације	Статус	Доказ имплементације и коментар
			преко HTTP контролних рута. Иако је ово решење функционално потпуно еквивалентно и тачно рефлектује све QoS/acks конфигурације, оно строго технички крши захтев пројекта. <i>Предложена корективна мера:</i> Уколико предметни професор инсистира на овој ставки као елиминационој, потребно је покренути један додатни контролни тест сет користећи стандардни k6 алат са одговарајућим екстензијама, чиме би се верификовало да се добијени резултати подударају са нашим генерисаним подацима, без икакве потребе за изменама у самој архитектури развијених микросервиса.